

Практическая работа №5

Выполнила: Ушакова Е.А. гр. ЗП-31

Все исходные файлы лабораторных работ лежат в папке репозитория по ссылке

<https://github.com/irbi42/uni/tree/main/vp/labs>

1) Реализация слотов экспорта и импорта таблицы в CSV. Данная функция открывает файл, задает поток вывода в этот файл, затем выгружает заголовки таблицы и все строки в соответствующем формате

```
88
89 bool ApplicationWindow::exportTableToCsv(QTableWidget *table, const QString &filename) {
90     QFile file(filename);
91     if (!file.open(QIODevice::WriteOnly | QIODevice::Text))
92         return false;
93     QTextStream out(&file);
94     QStringList headers;
95     for (int col = 0; col < table->columnCount(); ++col) {
96         headers << table->horizontalHeaderItem(col)->text();
97     }
98     out << headers.join(",") + "\n";
99     for (int row = 0; row < table->rowCount(); ++row) {
100         QStringList rowText;
101         for (int col = 0; col < table->columnCount(); ++col) {
102             QTableWidgetItem *item = table->item(row, col);
103             rowText << (item ? item->text() : "");
104         }
105         out << rowText.join(",") + "\n";
106     }
107     file.close();
108     return true;
109 }
110
```

Импорт происходит путем построчного считывания файла, где первая строка — заголовки, а все последующие — строки с данными, загружаемыми в таблицу.

```
111
112 bool ApplicationWindow::importCsvToTable(const QString &filename, QTableWidget *table) {
113     QFile file(filename);
114     if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
115         QMessageBox::critical(this, tr("Error"), tr("Could not open the file."));
116         return false;
117     }
118     QTextStream in(&file);
119     table->clearContents();
120     table->setRowCount(0);
121     table->setColumnCount(0);
122     int row = 0;
123     while (!in.atEnd()) {
124         QString line = in.readLine();
125         QStringList fields = line.split(',');
126         if (row == 0) {
127             table->setColumnCount(fields.size());
128             table->setHorizontalHeaderLabels(fields);
129         } else {
130             table->insertRow(table->rowCount());
131             for (int col = 0; col < fields.size(); ++col) {
132                 table->setItem(row - 1, col, new QTableWidgetItem(fields[col]));
133             }
134         }
135         row++;
136     }
137     file.close();
138     return true;
139 }
140
```

2) сохранение документа. Для начала проверяется наличие ссылки на документ, если альтернативная ссылка пуста, документ записывается по ссылке, хранящейся в документе. Затем получается ссылка на текущую таблицу, которая затем экспортируется с помощью функции, реализованной выше.

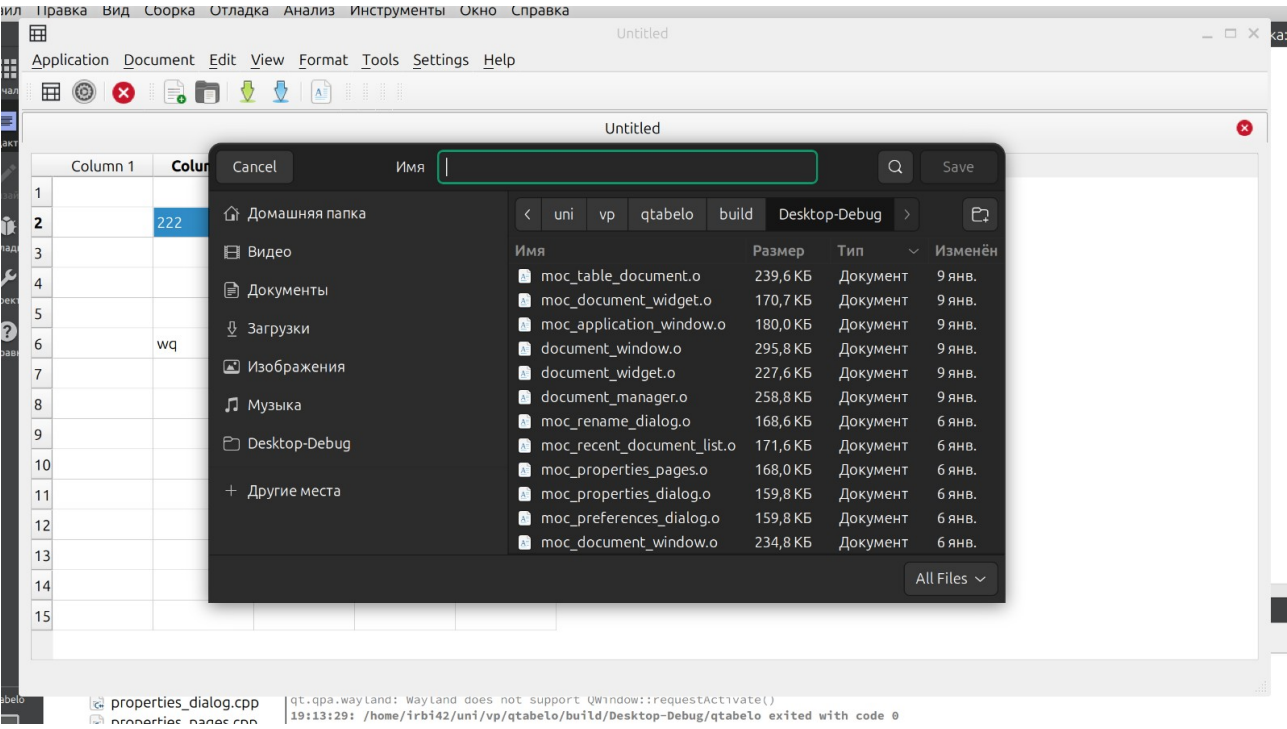
```
1055 }
1056
1057
1058 bool ApplicationWindow::saveDocument(DocumentWidget *document, const QUrl &altUrl)
1059 {
1060     // Q_UNUSED(resetModified());
1061     QUrl url = altUrl.isEmpty() ? document->url() : altUrl;
1062     if (!url.isValid()) {
1063         QMessageBox::critical(this, tr("Error"), tr("Invalid file location"));
1064     }
1065     QTableWidgetItem *table = nullptr;
1066     if (auto tabs = document->findChild<QTabWidget*>()) {
1067         table = qobject_cast<QTableWidgetItem*>(tabs->currentWidget());
1068     }
1069     if (!table) {
1070         QMessageBox::critical(this, tr("Error"), tr("No table data to save."));
1071         return false;
1072     }
1073     if (!exportTableToCsv(table, url.toLocalFile())) {
1074         QMessageBox::critical(this, tr("Error"), tr("Could not save the file."));
1075         return false;
1076     }
1077
1078     document->resetModified();
1079     document->setUrl(url);
1080
1081     m_recentDocuments->addUrl(url);
1082
1083     return true;
1084 }
1085
```

3) Загрузка документа. Функция получает на вход ссылку на документ. Затем импортирует файл в таблицу.

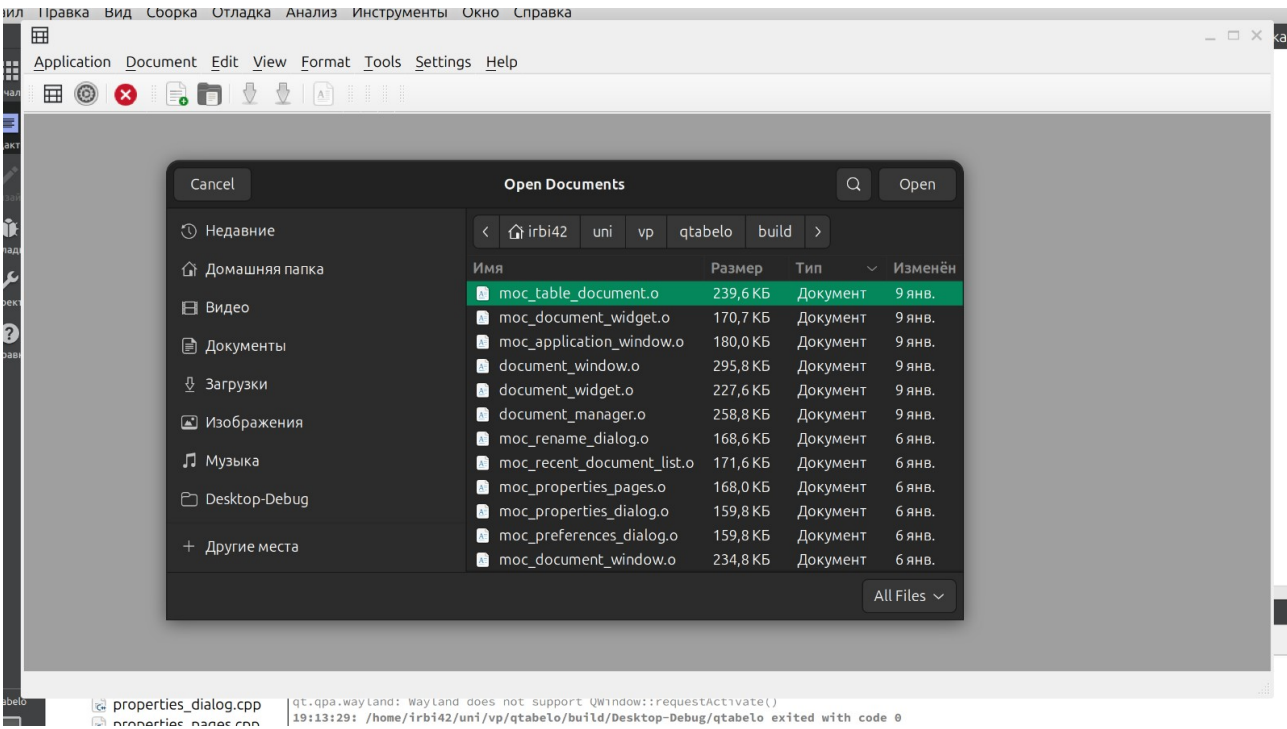
```
1024
1025 bool ApplicationWindow::loadDocument(const QUrl &url)
1026 {
1027     DocumentWidget *document = createDocument();
1028     if (!document) return false;
1029
1030     document->documentCountChanged(1);
1031
1032     QTableWidgetItem *table = nullptr;
1033     if (document->findChild<QTabWidget*>()) {
1034         QTabWidget *tabs = document->findChild<QTabWidget*>();
1035         table = qobject_cast<QTableWidgetItem*>(tabs->currentWidget());
1036     }
1037
1038     if (!table) {
1039         document->close();
1040         return false;
1041     }
1042
1043     if (!importCsvToTable(url.toLocalFile(), table)) {
1044         document->close();
1045         return false;
1046     }
1047
1048     document->show();
1049     document->setUrl(url);
1050     m_recentDocuments->addUrl(url);
1051
1052     documentCreated();
1053
1054     return true;

```

Результат работы
Сохранение:



Загрузка



Ответы на контрольные вопросы

Какие этапы включает загрузка данных из CSV в QTableWidgetItem?

Открытие файла через QFile, чтение построчно через QTextStream, разбиение строки по разделителю ',', определение заголовков из первой строки, установка количества столбцов, вставка строк и заполнение ячеек через setItem(), закрытие файла.

Какие классы в QTabelo отвечают за управление документами?

ApplicationWindow (главное окно), DocumentManager (управление вкладками/окнами), DocumentWidget (наследник TableDocument, хранит данные таблицы и состояние), DocumentWindow (обёртка для MDI).

Как связаны между собой классы ApplicationWindow, DocumentManager и DocumentWidget при работе с CSV?

ApplicationWindow содержит DocumentManager как центральный виджет. При открытии CSV вызывает createDocument(), который создаёт DocumentWidget. Затем вызывает importCsvToTable(), передавая QTableWidgetItem из DocumentWidget. При сохранении — exportTableToCsvs().

Какие сигналы и слоты задействованы при импорте/экспорте таблиц?

loadDocument() → importCsvToTable().
slotSave()/slotSaveAs() → saveDocument() → exportTableToCsvs().
Также сигналы modifiedChanged, urlChanged для обновления состояния.

Какие улучшения можно добавить в функциональность работы с CSV?

Поддержка разных разделителей (., ;, \t), обработка кавычек и экранирования.